



PYTHON PROGRAMMING

Chapter 4

Built-in Functions & Core Python Objects

Covering: Sets • None • Bool • 21 Built-in Functions

Chapter 4: Python Built-in Functions

Chapter Overview

This chapter introduces you to some of the most important building blocks of Python programming: the Set object, the None and Bool data types, and a rich set of built-in functions that Python provides out-of-the-box.

You do not need to import anything special to use these — they are always available, ready for you to use as soon as you open Python.

Learning Objectives

By the end of this chapter, you will be able to:

- Understand and work with Set, None, and Bool objects
- Use Python's built-in functions to perform common operations without writing them from scratch
- Apply functions like `input()`, `len()`, `type()`, `range()`, `map()`, `filter()`, `sorted()`, and many more
- Read and interpret function outputs with real code examples

Section 1: Introduction to Set, None, and Bool Objects

1.1 The Set Object

What is a Set?

A Set is a collection of unique items. Think of it like a bag where you cannot put two identical items — if you try to add a duplicate, Python simply ignores it.

A Set has three important characteristics:

- **Unordered:** Items in a set do not have a fixed position or index.
- **Unique:** No duplicate values are allowed.
- **Mutable:** You can add or remove items after creation.

Creating a Set

You create a set using curly braces {} or the set() constructor.

Example 1.1 — Creating Sets

```
# Method 1: Using curly braces
fruits = {"apple", "banana", "mango"}
print(fruits)

# Method 2: Using set() constructor
numbers = set([1, 2, 3, 4, 5])
print(numbers)

# Method 3: Creating an empty set (IMPORTANT!)
empty_set = set() # Correct way
not_a_set = {}    # This creates a dictionary, NOT a set!
```

Output

```
{'banana', 'apple', 'mango'}
{1, 2, 3, 4, 5}
```

Important Note

Notice that the order of items in the output may differ from the order you entered them. This is normal — sets are unordered, so Python does not guarantee any specific sequence.

Duplicate Values Are Automatically Removed

One of the most useful properties of a set is that it automatically eliminates duplicates:

Example 1.2 — Automatic Duplicate Removal

```
# Duplicates are removed automatically
student_ids = {101, 102, 103, 102, 101, 104}
print(student_ids)

# Practical use: find unique items from a list
scores = [85, 90, 85, 72, 90, 100, 72]
unique_scores = set(scores)
print(unique_scores)
```

Output

```
{101, 102, 103, 104}
{72, 85, 90, 100}
```

Common Set Operations

Sets support mathematical operations that are extremely useful in data comparison:

Example 1.3 — Set Operations

```
# Define two sets
set_a = {1, 2, 3, 4, 5}
set_b = {4, 5, 6, 7, 8}

# Union: all items from both sets
print("Union:", set_a | set_b)

# Intersection: items common to both sets
print("Intersection:", set_a & set_b)

# Difference: items in set_a but NOT in set_b
print("Difference (A - B):", set_a - set_b)

# Symmetric Difference: items in one set but NOT both
print("Symmetric Difference:", set_a ^ set_b)
```

Output

```
Union: {1, 2, 3, 4, 5, 6, 7, 8}
Intersection: {4, 5}
Difference (A - B): {1, 2, 3}
Symmetric Difference: {1, 2, 3, 6, 7, 8}
```

Adding and Removing Items

Example 1.4 — Modifying a Set

```
languages = {"Python", "Java", "C++"}

# Add a single item
languages.add("JavaScript")
print(languages)

# Remove an item (raises error if not found)
languages.remove("C++")
print(languages)

# Discard: removes item without error if it doesn't exist
languages.discard("Ruby") # No error, even though Ruby is not in the set
print(languages)
```

Output

```
{'Python', 'Java', 'C++', 'JavaScript'}
{'Python', 'Java', 'JavaScript'}
{'Python', 'Java', 'JavaScript'}
```

Set Methods — Quick Reference

Method	Description	Example
<code>add(x)</code>	Adds element <code>x</code> to the set	<code>s.add(5)</code>
<code>remove(x)</code>	Removes <code>x</code> ; error if not found	<code>s.remove(3)</code>
<code>discard(x)</code>	Removes <code>x</code> ; no error if missing	<code>s.discard(10)</code>
<code>pop()</code>	Removes a random item	<code>s.pop()</code>
<code>clear()</code>	Empties the entire set	<code>s.clear()</code>
<code>union(s2)</code>	Returns union of two sets	<code>s1.union(s2)</code>
<code>intersection(s2)</code>	Returns common elements	<code>s1.intersection(s2)</code>
<code>difference(s2)</code>	Returns elements in <code>s1</code> not in <code>s2</code>	<code>s1.difference(s2)</code>
<code>issubset(s2)</code>	Checks if <code>s1</code> is inside <code>s2</code>	<code>s1.issubset(s2)</code>
<code>issuperset(s2)</code>	Checks if <code>s1</code> contains <code>s2</code>	<code>s1.issuperset(s2)</code>

1.2 The None Object

What is None?

None is a special constant in Python that represents the absence of a value. It is similar to null in other programming languages. None is not zero, not False, not an empty string — it is literally nothing.

None belongs to its own data type called `NoneType`:

Example 1.5 — The None Object

```
# None represents "no value"
result = None
print(result)      # Outputs: None
print(type(result)) # Outputs: <class 'NoneType'>
```

Output

```
None
<class 'NoneType'>
```

When Does None Appear?

None appears in several common situations in Python:

Example 1.6 — None in Practice

```
# 1. A function that does not explicitly return a value
def greet(name):
    print("Hello,", name)
    # No return statement

result = greet("Alice")
print("Return value:", result) # Outputs: None

# 2. Initializing a variable with no value yet
user_input = None
# ... (later in the program, this will be assigned a real value)

# 3. Checking for None
if user_input is None:
    print("No input received yet")
```

Output

```
Hello, Alice
Return value: None
No input received yet
```

**Best Practice**

Always use `'is None'` or `'is not None'` to check for `None` — not `'== None'`. This is the Python-recommended style and is more accurate.

Correct: `if value is None:`

Incorrect: `if value == None:`

1.3 The Bool Object

What is a Bool (Boolean)?

A Boolean is a data type that can hold one of only two values: True or False. Python uses capital T and capital F — these are not strings, they are actual Boolean values.

Example 1.7 — Boolean Values

```
# Boolean values
is_logged_in = True
is_admin = False

print(is_logged_in)    # True
print(type(is_logged_in)) # <class 'bool'>
print(is_admin)        # False
```

Output

```
True
<class 'bool'>
False
```

Boolean from Comparisons

Most comparisons in Python produce a Boolean result:

Example 1.8 — Comparisons Produce Booleans

```
age = 20

print(age > 18)    # True — 20 is greater than 18
print(age == 18)   # False — 20 is not equal to 18
print(age != 18)   # True — 20 is not equal to 18
print(age >= 20)   # True — 20 is greater than or equal to 20
print(age < 10)    # False — 20 is not less than 10
```

Output

```
True
False
True
True
False
```

Bool and Numbers

In Python, bool is a subtype of int. True equals 1 and False equals 0. This allows you to use Booleans in arithmetic:

Example 1.9 — Booleans in Arithmetic

```
print(True + True) # 2
print(True + False) # 1
print(False + False) # 0

# Practical use: counting how many conditions are True
scores = [85, 92, 45, 78, 60]
passed = sum(score >= 60 for score in scores)
print("Students who passed:", passed)
```

Output

```
2
1
0
Students who passed: 4
```

Truthy and Falsy Values

Every value in Python has an implicit Boolean meaning. Values that behave like False are called Falsy; everything else is Truthy.

Falsy Values (behave like False)	Truthy Values (behave like True)
False	True
0, 0.0, 0j	Any non-zero number (1, -5, 3.14)
None	Any non-None value
"" (empty string)	"hello" (any non-empty string)
[] (empty list)	[1, 2, 3] (non-empty list)
{ } (empty dict/set)	{"a": 1} (non-empty dict)

Example 1.10 — Truthy and Falsy

```
# Using truthy/falsy in conditions
name = ""

if name:
    print("Name is provided:", name)
else:
    print("Name is empty — please enter your name")

# With a non-empty value
name = "Alice"
if name:
    print("Name is provided:", name)
```

Output

Name is empty — please enter your name
Name is provided: Alice

ETHANS
CLOUD & AI ACADEMY



Section 2: Built-in Functions in Python

Python comes with a large set of built-in functions — ready-to-use tools that are always available without importing any library. They handle the most common programming tasks: reading input, measuring size, converting types, sorting data, and much more.

What Makes a Function 'Built-in'?

A built-in function is pre-installed with Python. You do not need to write it yourself or import it from a library — it is always available. Examples: `print()`, `len()`, `range()`, `type()`.

This is different from library functions (like `math.sqrt()`) which require an import statement first.

2.1 `input()` — Reading User Input

What it does: Pauses the program and waits for the user to type something. Whatever the user types is returned as a string.

Syntax: `variable = input(prompt)`

Example 2.1 — Basic Input

```
# Basic usage
name = input("Enter your name: ")
print("Hello,", name)
```

Important: `input()` Always Returns a String

Even if the user types a number like 25, `input()` gives you the string "25", not the integer 25. You must convert it using `int()` or `float()` if you need to do arithmetic.

Example 2.2 — Converting `input()` to Numbers

```
# Incorrect: trying to add a string and a number
age_str = input("Enter your age: ") # User types: 25
# print(age_str + 5)                # This would cause a TypeError!

# Correct: convert to integer first
age = int(input("Enter your age: ")) # User types: 25
print("In 5 years, you will be:", age + 5)

# Working with decimal input
height = float(input("Enter your height in meters: ")) # e.g., 1.75
print("Your height:", height, "m")
```

Output

In 5 years, you will be: 30
Your height: 1.75 m

2.2 len() — Finding the Length

What it does: Returns the number of items in a sequence or collection — such as a string, list, tuple, set, or dictionary.

Syntax: `len(object)`

Example 2.3 — len() with Different Types

```
# len() with different data types
name = "Python"
print(len(name))      # Number of characters in the string

cities = ["Delhi", "Mumbai", "Bangalore"]
print(len(cities))    # Number of items in the list

student_ids = {101, 102, 103, 104}
print(len(student_ids)) # Number of items in the set

info = {"name": "Alice", "age": 22, "grade": "A"}
print(len(info))      # Number of key-value pairs in the dictionary
```

Output

6
3
4
3

Example 2.4 — Using len() for Validation

```
# Practical example: validating password length
password = input("Create a password (min 8 characters): ")
if len(password) < 8:
    print("Password too short. Please use at least 8 characters.")
else:
    print("Password accepted!")
```

Output

Password too short. Please use at least 8 characters.

2.3 type() — Checking the Data Type

What it does: Returns the data type (class) of the given object. Extremely useful for debugging when you are unsure what kind of value you are working with.

Syntax: `type(object)`

Example 2.5 — type() with All Common Types

```
# type() with various data types
print(type(42))      # <class 'int'>
print(type(3.14))    # <class 'float'>
print(type("Hello")) # <class 'str'>
print(type(True))    # <class 'bool'>
print(type(None))    # <class 'NoneType'>
print(type([1, 2, 3])) # <class 'list'>
print(type((1, 2, 3))) # <class 'tuple'>
print(type({1, 2, 3})) # <class 'set'>
print(type({"a": 1})) # <class 'dict'>
```

Output

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
<class 'list'>
<class 'tuple'>
<class 'set'>
<class 'dict'>
```

Example 2.6 — Using type() to Diagnose Issues

```
# Practical: using type() in conditions
value = input("Enter a value: ") # Returns a string
print("You entered:", value)
print("Its type:", type(value))

# Comparing type
if type(value) == str:
    print("It is a string — conversion may be needed for arithmetic")
```

2.4 abs() — Absolute Value

What it does: Returns the absolute value of a number — that is, the number without its negative sign. Positive numbers remain unchanged.

Syntax: `abs(number)`

Example 2.7 — `abs()` in Action

```
print(abs(-10)) # 10
print(abs(10))  # 10
print(abs(-3.75)) # 3.75
print(abs(0))   # 0

# Practical: calculating temperature difference
temp_today = 15
temp_tomorrow = 22
difference = abs(temp_today - temp_tomorrow)
print("Temperature difference:", difference, "degrees")
```

Output

```
10
10
3.75
0
Temperature difference: 7 degrees
```

2.5 `pow()` — Power / Exponentiation

What it does: Calculates the power of a number (base raised to an exponent). It also accepts an optional third argument for modulus.

Syntax: `pow(base, exponent)` or `pow(base, exponent, mod)`

Example 2.8 — `pow()`

```
print(pow(2, 10)) # 2 to the power 10 = 1024
print(pow(3, 3))  # 3 to the power 3 = 27
print(pow(5, 0))  # Any number to the power 0 = 1
print(pow(4, 0.5)) # Square root of 4 = 2.0

# With modulus: pow(base, exp, mod) = (base ** exp) % mod
print(pow(2, 10, 100)) # 1024 % 100 = 24
```

Output

```
1024
27
1
2.0
24
```

**Tip**

`pow(2, 10)` is equivalent to writing `2 ** 10` in Python. The `**` operator is shorthand. However, `pow()` with three arguments is more efficient for large numbers.

2.6 `min()` and `max()` — Minimum and Maximum

What they do: `min()` returns the smallest value; `max()` returns the largest. Both work with a sequence of numbers or individual arguments.

Syntax: `min(iterable)` or `min(a, b, c, ...)` (same for `max`)

Example 2.9 — `min()` and `max()`

```
# With a list
exam_scores = [72, 85, 61, 90, 78, 55, 88]
print("Lowest score:", min(exam_scores))
print("Highest score:", max(exam_scores))

# With individual values
print(min(10, 25, 3, 47)) # 3
print(max(10, 25, 3, 47)) # 47

# With strings: alphabetical comparison
fruits = ["mango", "apple", "banana"]
print(min(fruits)) # apple (alphabetically first)
print(max(fruits)) # mango (alphabetically last)
```

Output

```
Lowest score: 55
Highest score: 90
3
47
apple
mango
```

2.7 `sum()` — Calculating the Total

What it does: Adds all numeric items in an iterable (like a list or tuple) and returns the total. You can also provide a starting value.

Syntax: `sum(iterable, start=0)`

Example 2.10 — `sum()`

```
# Basic usage
marks = [85, 92, 78, 88, 95]
total = sum(marks)
print("Total marks:", total)

# Calculate average
average = sum(marks) / len(marks)
print("Average:", average)

# With a starting value (adds 100 to the total)
total_with_bonus = sum(marks, 100)
print("Total with 100 bonus:", total_with_bonus)
```

Output

```
Total marks: 438
Average: 87.6
Total with 100 bonus: 538
```

2.8 range() — Generating Number Sequences

What it does: Generates a sequence of numbers. Widely used in for loops to repeat an action a specific number of times.

Three forms:

- **range(stop)** → 0, 1, 2, ..., stop-1
- **range(start, stop)** → start, start+1, ..., stop-1
- **range(start, stop, step)** → from start, counting by step

Example 2.11 — range() Forms

```
# Form 1: range(stop)
for i in range(5):
    print(i, end=" ")
print()

# Form 2: range(start, stop)
for i in range(1, 6):
    print(i, end=" ")
print()

# Form 3: range(start, stop, step)
for i in range(0, 20, 5):
    print(i, end=" ")
print()

# Counting down with negative step
for i in range(10, 0, -2):
```

```
print(i, end=" ")
print()
```

Output

```
0 1 2 3 4
1 2 3 4 5
0 5 10 15
10 8 6 4 2
```

Example 2.12 — Practical Use of range()

```
# Practical: generating a multiplication table
n = 7
print(f"Multiplication Table of {n}")
print("-" * 25)
for i in range(1, 11):
    print(f"{n} x {i:2d} = {n * i}")
```

Output

```
Multiplication Table of 7
-----
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

2.9 enumerate() — Index + Value Together

What it does: Loops over an iterable and gives you both the index (position number) and the value at each step. Saves you from writing a separate counter variable.

Syntax: `enumerate(iterable, start=0)`

Example 2.13 — enumerate() vs Manual Indexing

```
# Without enumerate (old way — more code needed)
fruits = ["apple", "banana", "mango"]
index = 0
for fruit in fruits:
    print(index, fruit)
    index += 1
```

```
print()

# With enumerate (clean and Pythonic)
for index, fruit in enumerate(fruits):
    print(index, fruit)
```

Output

```
0 apple
1 banana
2 mango

0 apple
1 banana
2 mango
```

Example 2.14 — enumerate() with Custom Start

```
# Starting index from 1 instead of 0
students = ["Alice", "Bob", "Carol", "David"]
for rank, name in enumerate(students, start=1):
    print(f"Rank {rank}: {name}")
```

Output

```
Rank 1: Alice
Rank 2: Bob
Rank 3: Carol
Rank 4: David
```

2.10 zip() — Pairing Multiple Lists

What it does: Combines multiple iterables (lists, tuples, etc.) element by element. It produces pairs (or tuples) from corresponding positions. Stops at the shortest iterable.

Syntax: `zip(iterable1, iterable2, ...)`

Example 2.15 — zip() Basic Pairing

```
# Pairing two lists
names = ["Alice", "Bob", "Carol"]
scores = [88, 75, 92]

for name, score in zip(names, scores):
    print(f"{name}: {score}")
```


Output

```
Alice: 88
Bob: 75
Carol: 92
```

Example 2.16 — zip() Advanced Usage

```
# Creating a dictionary from two lists
keys = ["name", "age", "city"]
values = ["Alice", 22, "Mumbai"]

person = dict(zip(keys, values))
print(person)

# Zipping three lists
subjects = ["Math", "Science", "English"]
teacher = ["Mr. Sharma", "Ms. Patel", "Mr. Khan"]
room = [101, 202, 303]

for sub, tch, rm in zip(subjects, teacher, room):
    print(f"{sub} — {tch} — Room {rm}")
```

Output

```
{'name': 'Alice', 'age': 22, 'city': 'Mumbai'}
Math — Mr. Sharma — Room 101
Science — Ms. Patel — Room 202
English — Mr. Khan — Room 303
```

2.11 map() — Applying a Function to Every Item

What it does: Applies a given function to every item in an iterable and returns the results. It replaces the need to write a loop just to transform each element.

Syntax: `map(function, iterable)`

Example 2.17 — map() with Built-in and Lambda Functions

```
# Convert a list of strings to integers
str_nums = ["1", "2", "3", "4", "5"]
int_nums = list(map(int, str_nums))
print(int_nums)

# Square every number in a list
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x ** 2, numbers))
print(squares)

# Convert Celsius to Fahrenheit
```

```
celsius = [0, 20, 37, 100]
fahrenheit = list(map(lambda c: (c * 9/5) + 32, celsius))
print(fahrenheit)
```

Output

```
[1, 2, 3, 4, 5]
[1, 4, 9, 16, 25]
[32.0, 68.0, 98.6, 212.0]
```

**What is a lambda?**

A lambda is a small, one-line anonymous function. Instead of defining a full function with `def`, you write:
`lambda x: expression`

`lambda x: x ** 2` is the same as:

```
def square(x):
    return x ** 2
```

2.12 filter() — Selecting Items That Meet a Condition

What it does: Goes through an iterable and keeps only the items for which a given function returns `True`. Think of it as a sieve.

Syntax: `filter(function, iterable)`

Example 2.18 — filter() Examples

```
# Keep only even numbers
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print("Even numbers:", evens)

# Keep only passing scores (>= 60)
scores = [45, 78, 33, 90, 55, 62, 88]
passing = list(filter(lambda s: s >= 60, scores))
print("Passing scores:", passing)

# Keep only non-empty strings
names = ["Alice", "", "Bob", " ", "Carol", ""]
valid_names = list(filter(lambda n: n.strip() != "", names))
print("Valid names:", valid_names)
```

Output

```
Even numbers: [2, 4, 6, 8, 10]
Passing scores: [78, 90, 62, 88]
```

Valid names: ['Alice', 'Bob', 'Carol']

map()	filter()
Transforms every element	Selects a subset of elements
Output has the same count as input	Output may have fewer items than input
Function can return any value	Function must return True or False
Use when: changing all values	Use when: removing some values

2.13 sorted() — Sorting Any Sequence

What it does: Returns a new sorted list from the items in an iterable. The original list is NOT modified. Works with any comparable data type.

Syntax: `sorted(iterable, key=None, reverse=False)`

Example 2.19 — sorted() with Various Options

```
# Basic sorting
numbers = [5, 2, 9, 1, 7, 3]
sorted_nums = sorted(numbers)
print("Sorted:", sorted_nums)
print("Original:", numbers) # Unchanged!

# Sort in descending order
desc = sorted(numbers, reverse=True)
print("Descending:", desc)

# Sort strings alphabetically
cities = ["Mumbai", "Delhi", "Ahmedabad", "Bangalore"]
print(sorted(cities))

# Sort by string length using key
words = ["banana", "fig", "apple", "kiwi", "pear"]
by_length = sorted(words, key=len)
print("By length:", by_length)
```

Output

```
Sorted: [1, 2, 3, 5, 7, 9]
Original: [5, 2, 9, 1, 7, 3]
Descending: [9, 7, 5, 3, 2, 1]
['Ahmedabad', 'Bangalore', 'Delhi', 'Mumbai']
By length: ['fig', 'fig', 'kiwi', 'pear', 'apple', 'banana']
```

Example 2.20 — Sorting Dictionaries

```
# Sorting a list of dictionaries
students = [
    {"name": "Alice", "grade": 88},
    {"name": "Bob", "grade": 75},
    {"name": "Carol", "grade": 92},
]
by_grade = sorted(students, key=lambda s: s["grade"], reverse=True)
for s in by_grade:
    print(f'{s["name"]}: {s["grade"]}')
```

Output

```
Carol: 92
Alice: 88
Bob: 75
```

2.14 reversed() — Reversing a Sequence

What it does: Returns an iterator that goes through the elements in reverse order. To see all elements, convert it to a list.

Syntax: `reversed(sequence)`

Example 2.21 — reversed()

```
# Reverse a list
numbers = [1, 2, 3, 4, 5]
rev = list(reversed(numbers))
print(rev)

# Reverse a string's characters
word = "Python"
rev_word = "".join(reversed(word))
print(rev_word)

# Reverse in a for loop (no conversion needed)
fruits = ["apple", "banana", "mango"]
print("In reverse order:")
for fruit in reversed(fruits):
    print("-", fruit)
```

Output

```
[5, 4, 3, 2, 1]
nohtyP
In reverse order:
- mango
```

```
- banana  
- apple
```

sorted() vs reversed()

`sorted()` → Returns a new sorted list (from smallest to largest by default)

`reversed()` → Returns the sequence in backwards order (does NOT sort)

Example: `sorted([3,1,2])` → [1, 2, 3]

`list(reversed([3,1,2]))` → [2, 1, 3]

2.15 `all()` and `any()` — Checking Conditions Across Items

`all()`: Returns True if every item in the iterable is truthy (or the iterable is empty).

`any()`: Returns True if at least one item in the iterable is truthy.

Syntax: `all(iterable)` | `any(iterable)`

Example 2.22 — `all()` and `any()`

```
# all() — checks that every item meets the condition
```

```
scores = [75, 80, 90, 65, 72]
```

```
all_passed = all(s >= 60 for s in scores)
```

```
print("Did everyone pass?", all_passed)
```

```
scores2 = [75, 80, 45, 65, 72] # 45 is failing
```

```
all_passed2 = all(s >= 60 for s in scores2)
```

```
print("Did everyone pass?", all_passed2)
```

```
# any() — checks if at least one item meets the condition
```

```
any_distinction = any(s >= 90 for s in scores)
```

```
print("Any distinction?", any_distinction)
```

```
# Checking boolean list
```

```
permissions = [True, True, True]
```

```
print("All permissions granted?", all(permissions))
```

```
permissions2 = [True, False, True]
```

```
print("Any permission denied?", any(not p for p in permissions2))
```

Output

```
Did everyone pass? True
```

```
Did everyone pass? False
```

```
Any distinction? True
```

```
All permissions granted? True
```

```
Any permission denied? True
```

2.16 id() — Memory Address of an Object

What it does: Returns the unique identity (memory address) of an object. Every object in Python has a unique id while it exists in memory.

Syntax: `id(object)`

Example 2.23 — id()

```
x = 100
y = 100
z = [1, 2, 3]

print("id of x:", id(x))
print("id of y:", id(y))
print("id of z:", id(z))
print()

# Small integers are cached — x and y may share same id
print("x and y same object?", x is y)

# Lists always get new memory — different id even with same content
a = [1, 2, 3]
b = [1, 2, 3]
print("a and b same object?", a is b)    # False
print("a and b equal?", a == b)         # True
```

Output

```
id of x: 140721234567890
id of y: 140721234567890
id of z: 2048123456789
x and y same object? True
a and b same object? False
a and b equal? True
```



Note on id() Values

The exact id numbers will be different on your computer — they depend on your system's memory allocation. What matters is whether two variables have the same id (same object) or different ids (different objects).

2.17 help() — Built-in Documentation

What it does: Displays the documentation for any Python object, function, module, or keyword. It is like having the Python manual available directly inside your program.

Syntax: `help(object)`

Example 2.24 — Using `help()`

```
# Get help on a built-in function
help(len)

# Get help on a data type
help(str)

# Get help on a module (after import)
import math
help(math.sqrt)
```



When to Use `help()`

Use `help()` when you are unsure about:

- What arguments a function accepts
- What a function returns
- Available methods on an object

It is the first place to look before searching the web!

2.18 `dir()` — Listing All Attributes and Methods

What it does: Returns a list of all attributes and methods available on an object. Useful for exploring what you can do with a value.

Syntax: `dir(object)`

Example 2.25 — `dir()` on a String

```
# What methods does a string have?
name = "Python"
print(dir(name))

# Filter out 'dunder' methods (those with __) for cleaner output
methods = [m for m in dir(name) if not m.startswith("__")]
print(methods)
```

Output

```
['capitalize', 'casefold', 'center', 'count', 'encode', 'endswith',
'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum',
'isalpha', 'isascii', 'isdecimal', 'isdigit', 'isidentifier',
```

```
'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle',
'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketrans',
'partition', 'removeprefix', 'removesuffix', 'replace', 'rfind',
'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip', 'split',
'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'upper', 'zfill']
```

2.19 isinstance() — Type Checking

What it does: Checks whether an object is an instance of a specified class (or one of several classes). Returns True or False. This is the preferred way to check types in Python.

Syntax: `isinstance(object, classinfo)`

Example 2.26 — isinstance() for Type Safety

```
# Basic type checking
x = 42
print(isinstance(x, int))    # True
print(isinstance(x, float))  # False
print(isinstance(x, str))    # False

name = "Alice"
print(isinstance(name, str)) # True

# Check against multiple types at once
value = 3.14
print(isinstance(value, (int, float))) # True — it matches float

# Practical: safe arithmetic
def add_numbers(a, b):
    if not isinstance(a, (int, float)):
        print("Error: 'a' must be a number")
        return None
    if not isinstance(b, (int, float)):
        print("Error: 'b' must be a number")
        return None
    return a + b

print(add_numbers(10, 5))    # 15
print(add_numbers("ten", 5)) # Error
```

Output

```
True
False
False
True
True
15
Error: 'a' must be a number
None
```


type() check	isinstance() check
<code>type(x) == int</code>	<code>isinstance(x, int)</code>
Only exact match — subclass not included	Includes subclasses — more flexible
<code>type(True) == int</code> → False	<code>isinstance(True, int)</code> → True (bool is subtype of int)
Cannot check multiple types at once	<code>isinstance(x, (int, float))</code> — checks both

ETHANS
CLOUD & AI ACADEMY



Chapter Summary

Section 1 — Set, None, Bool Quick Reference

Object	Key Properties	Common Use
set	Unordered, unique, mutable	Removing duplicates, set math
None	Represents absence of value	Default return, empty state
bool	True or False only	Conditions, comparisons

Section 2 — Built-in Functions Quick Reference

Function	Purpose	Returns
input()	Read user input from keyboard	str
len()	Count items in a sequence	int
type()	Get the data type of an object	type object
abs()	Absolute value (remove negative sign)	int / float
pow()	Raise a number to a power	int / float
min()	Smallest value in a collection	Any comparable type
max()	Largest value in a collection	Any comparable type
sum()	Add all numeric values	int / float
range()	Generate a number sequence	range object
enumerate()	Loop with index and value	enumerate object
zip()	Pair items from multiple iterables	zip object
map()	Apply function to every item	map object
filter()	Keep items matching a condition	filter object
sorted()	Return new sorted list	list
reversed()	Return iterator in reverse order	reversed object
all()	True if all items are truthy	bool
any()	True if any item is truthy	bool
id()	Memory address of an object	int
help()	Show documentation for an object	None (prints docs)

<code>dir()</code>	List all attributes / methods	<code>list</code>
<code>isinstance()</code>	Check if object matches a type	<code>bool</code>

 Chapter Complete

You have now covered all the foundational Python objects and built-in functions covered in this chapter.

The next chapter will build on these tools as you learn to write your own functions and work with more complex data structures.

ETHANS
CLOUD & AI ACADEMY

